

Mini-projet GLPI — CY Tech

Rapport de projet Refonte Oracle de la base de données GLPI

Gestion du parc informatique multi-sites : Cergy et Pau

Bases de données avancées — GIA1 ING2

Année 2025-2026 - Duc Duy HUYNH, Axel COTTRANT, Alice FRANCO, Ostrean BONNET

Sujet	Repenser une partie de la BDD de GLPI pour améliorer la robustesse, les performances et la gestion multi-sites.
Périmètre	Matériels informatiques, utilisateurs, entités/sites et structure réseau.
Technologies	Oracle SQL, PL/SQL, tablespaces, rôles, clusters, index, vues, vues matérialisées, database links.
Dépôt	https://github.com/Axelcrt/bdd.git

Sommaire

1. 1. Introduction et objectifs du projet
2. 2. Démarche de reverse engineering
3. 3. Limites identifiées dans le modèle GLPI analysé
4. 4. Nouvelle architecture Oracle proposée
5. 5. Modélisation relationnelle, contraintes et types de données
6. 6. Automatisation PL/SQL : triggers, procédures, fonctions et package
7. 7. Vues, vues matérialisées et base de données répartie
8. 8. Optimisation : clusters, index et plans d'exécution
9. 9. Jeu de test et validation des performances
10. 10. Bilan, limites et améliorations possibles
11. 11. Conclusion

Ce sommaire est statique afin de conserver une mise en page stable dans un document modifiable. Les titres principaux reprennent les attendus du sujet et l'ordre d'exécution des scripts du dépôt.

1. Introduction et objectifs du projet

Le projet porte sur la refonte d'une partie de la base de données de GLPI, outil de gestion de parc informatique, dans le contexte du parc de CY Tech réparti sur deux sites : Cergy et Pau. L'objectif n'est pas de reproduire l'ensemble de GLPI, qui est un logiciel complet couvrant de nombreuses fonctionnalités, mais de retravailler un périmètre cohérent : les matériels informatiques, les utilisateurs, les entités organisationnelles et les informations liées au réseau.

Le sujet impose de mobiliser les principales notions de bases de données avancées : utilisateurs, rôles, tablespaces, clusters, index, vues, PL/SQL, bases de données réparties et analyse de plans d'exécution. Le livrable doit donc montrer à la fois une démarche de conception, une implémentation technique et une validation par des tests de performance.

La démarche suivie dans ce projet est volontairement structurée autour d'un axe simple : partir des limites du modèle GLPI analysé, proposer une architecture Oracle plus robuste, puis démontrer concrètement les apports par les scripts SQL et PL/SQL. Le dépôt est organisé dans cet ordre : reverse engineering, création des tablespaces et rôles, DDL des tables, vues, PL/SQL, jeu de test et tests de performance.

Élément	Choix retenu
SGBD cible	Oracle Database
Périmètre métier	Matériels, utilisateurs, sites, réseau
Sites traités	Cergy et Pau
Objectif principal	Robustesse, maintenabilité, gestion multi-sites et optimisation des requêtes
Validation	Jeu de test généré en PL/SQL et analyse avec EXPLAIN PLAN / DBMS_XPLAN

2. Démarche de reverse engineering

La première étape du projet consiste à analyser le schéma GLPI existant. Cette étape est essentielle car elle justifie les choix effectués ensuite dans la nouvelle base Oracle. Le reverse engineering ne sert donc pas seulement à lister les tables, mais à comprendre les limites du modèle initial et à définir les axes d'amélioration.

Le fichier 01_reverse_engineering.sql documente cette analyse. Le périmètre retenu se concentre sur les tables liées aux entités, utilisateurs, ordinateurs, équipements réseau, ports réseau, adresses IP, sous-réseaux, VLANs et composants mémoire. Ces éléments couvrent le cœur du besoin demandé : gérer un parc informatique multi-sites avec des informations matérielles et réseau.

Les tables GLPI analysées montrent une logique très orientée application. Dans le modèle étudié, de nombreuses relations sont portées par des colonnes d'identifiants sans contrainte de clé étrangère au niveau de la base. Cette approche peut fonctionner si l'application contrôle parfaitement toutes les opérations, mais elle fragilise la cohérence dès qu'une suppression, une insertion ou une modification est mal gérée.

Table GLPI étudiée	Rôle dans le périmètre
glpi_entities	Sites et entités organisationnelles, notamment Cergy et Pau
glpi_users	Utilisateurs, informations de connexion et rattachement à une entité
glpi_computers	Ordinateurs du parc informatique
glpi_networkequipments	Switchs, routeurs, bornes Wi-Fi et autres équipements réseau

glpi_networkports	Ports réseau rattachés aux ordinateurs ou équipements
glpi_ipaddresses / glpi_ipnetworks	Adresses IP et sous-réseaux
glpi_items_devicememories	Modules mémoire associés aux machines

3. Limites identifiées dans le modèle GLPI analysé

L'analyse fait ressortir plusieurs limites structurelles. La première concerne l'intégrité référentielle. Dans le modèle analysé, les relations entre les tables sont principalement gérées par le code applicatif. Une base de données robuste doit pourtant pouvoir garantir elle-même les relations essentielles entre les entités, les utilisateurs, les machines et leurs composants.

La deuxième limite concerne la gestion multi-sites. GLPI distingue les entités à l'aide d'une colonne `entities_id`, ce qui permet une séparation logique. Cependant, cette approche ne correspond pas à une vraie séparation physique des données entre Cergy et Pau. Pour un projet centré sur les bases de données réparties, il fallait aller plus loin : isoler les données par site et prévoir une logique d'accès inter-sites.

La troisième limite concerne l'exploitation des mécanismes avancés du SGBD. Le modèle initial repose surtout sur la logique applicative. Il n'exploite pas directement les concepts demandés dans le sujet : tablespaces, rôles Oracle, triggers, procédures, fonctions, vues matérialisées, database links ou plans de requêtes. Le projet consiste donc à déplacer une partie de la robustesse vers la base elle-même.

Limite constatée	Conséquence	Réponse proposée
Absence de vraies clés étrangères	Risque de données orphelines	Contraintes FOREIGN KEY et règles de suppression contrôlées
Gestion multi-sites logique uniquement	Pas d'isolation physique Cergy/Pau	Tables et tablespaces séparés par site
Types parfois imprécis	Contrôles difficiles sur RAM, fréquence, dates	NUMBER, DATE, VARCHAR2 et contraintes CHECK
Peu d'automatisation en base	Logique métier dépendante de l'application	Triggers, procédures, fonctions et package PL/SQL
Accès globaux coûteux	Requêtes multi-sites potentiellement lourdes	Vues classiques et vues matérialisées

4. Nouvelle architecture Oracle proposée

La nouvelle architecture s'appuie sur Oracle afin de mettre en œuvre les concepts avancés du cours.

L'organisation générale repose sur sept tablespaces dédiés. Deux tablespaces sont réservés aux matériels des deux sites : `TS_MATERIELS_CERGY` et `TS_MATERIELS_PAU`. Les autres tablespaces séparent les utilisateurs, le réseau, les index, l'audit et les vues matérialisées.

Cette séparation n'est pas uniquement esthétique. Elle permet de mieux administrer la base, d'isoler les volumes par domaine, d'anticiper les sauvegardes ciblées et de montrer une vraie logique multi-sites. Dans une architecture réelle, ce type d'organisation facilite également la supervision de l'espace disque et la maintenance des objets les plus volumineux.

La sécurité est également renforcée par des rôles Oracle. Le projet définit un rôle administrateur, un rôle technicien, un rôle responsable de site, un rôle utilisateur en lecture seule et un rôle auditeur. Cette approche ajoute une couche de gouvernance côté SGBD, en complément de la sécurité applicative. Un profil de sécurité `GLPI_SECURITE` encadre les connexions avec un verrouillage après plusieurs tentatives échouées et une durée de vie du mot de passe.

```
CREATE ROLE ROLE_ADMIN_GLPI;
CREATE ROLE ROLE_TECHNICIEN;
CREATE ROLE ROLE_RESP_SITE;
CREATE ROLE ROLE_UTILISATEUR;
CREATE ROLE ROLE_AUDITEUR;
```

Tablespace	Rôle
TS_USERS	Données communes : utilisateurs, entités, profils, localisations
TS_MATERIELS_CERGY	Matériels et composants du site de Cergy
TS_MATERIELS_PAU	Matériels et composants du site de Pau
TS_RESEAU	Équipements réseau, ports, IP, VLANs et sous-réseaux
TS_INDEX	Index B-tree, bitmap et index de clusters
TS_AUDIT	Table d'audit et journalisation métier
TS_MVIEWS	Vues matérialisées utilisées pour la réplication et le reporting

5. Modélisation relationnelle, contraintes et types de données

Le fichier 03_ddl_tables_clusters_index.sql contient le cœur de la structure relationnelle. Il crée les séquences, les clusters, les tables de référence, les tables de matériels par site, les composants et les index. L'un des choix structurants est de remplacer l'auto-incrémentation MySQL par des séquences Oracle, ensuite utilisées par des triggers d'insertion.

La table COMPUTERS_CERGY constitue un exemple représentatif. Elle contient les informations principales d'un ordinateur du site de Cergy : identifiant, entité, nom, numéro de série, utilisateur, localisation, système d'exploitation, modèle, fabricant, RAM déclarée, état et indicateur de suppression logique. Contrairement au modèle initial, les relations importantes sont encadrées par des clés étrangères et des contraintes.

Les types de données ont également été adaptés. Les valeurs numériques comme la RAM, les fréquences ou les identifiants sont stockées en NUMBER. Les dates de création et de modification sont stockées en DATE. Les booléens applicatifs sont représentés par des colonnes numériques contrôlées par des CHECK. L'objectif est de réduire les valeurs incohérentes et de clarifier le modèle.

```
CREATE SEQUENCE SEQ_COMPUTERS_C START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE SEQ_COMPUTERS_P START WITH 1 INCREMENT BY 1;

-- Exemple simplifié
CREATE TABLE COMPUTERS_CERGY (
  id NUMBER(10) NOT NULL,
  entities_id NUMBER(10) NOT NULL,
  name VARCHAR2(255) NOT NULL,
  ram_total NUMBER(10),
  is_deleted NUMBER(1) DEFAULT 0 CHECK (is_deleted IN (0,1)),
  date_creation DATE DEFAULT SYSDATE,
  date_mod DATE,
  CONSTRAINT PK_COMPUTERS_C PRIMARY KEY (id)
);
```

Cette modélisation permet de faire porter la cohérence sur la base elle-même. Par exemple, une machine doit appartenir à une entité existante, une localisation doit être rattachée à un site, et un composant ne doit pas être laissé sans machine de référence. La base devient donc un acteur de contrôle, et non un simple espace de stockage.

6. Automatisation PL/SQL : triggers, procédures, fonctions et package

La partie PL/SQL est l'un des apports les plus importants du projet. Elle montre que la base n'est pas seulement un ensemble de tables, mais qu'elle peut appliquer des règles métier, automatiser des traitements et garantir une partie de l'intégrité des données. Le fichier 05_plsql_triggers_procedures.sql regroupe les triggers, procédures, fonctions et le package.

Les triggers d'insertion permettent d'utiliser les séquences Oracle pour générer automatiquement les identifiants. D'autres triggers assurent l'audit des opérations, la mise à jour de dates de modification, la cohérence de la RAM ou la propagation du soft-delete. L'audit est particulièrement utile : une modification sur une machine est enregistrée dans AUDIT_LOG, ce qui permet de garder une trace des opérations sensibles.

Le trigger de vérification de la RAM illustre une règle métier concrète. Lorsqu'un module mémoire est ajouté ou modifié, la base peut recalculer la RAM réelle d'une machine à partir de ses composants. Si l'écart avec la RAM déclarée dépasse un seuil, l'information peut être corrigée et l'action auditée. Ce choix montre l'intérêt de centraliser certaines règles proches de la donnée.

Les procédures complètent cette logique. PRC_RAPPORT_INVENTAIRE génère un rapport d'inventaire à l'aide de curseurs. PRC_SEARCH_COMPUTERS expose un SYS_REFCURSOR pour filtrer les machines. PRC_TRANSFER_COMPUTER représente une opération transactionnelle : le transfert d'une machine entre sites doit être complet ou annulé par ROLLBACK en cas d'erreur. Enfin, PRC_REFRESH_MVIEWS permet de rafraîchir les vues matérialisées.

Objet PL/SQL	Utilité
TRG_AUDIT_COMPUTERS_CERGY	Historiser les insertions, modifications et suppressions sur les machines
TRG_CHECK_RAM_CERGY	Comparer la RAM déclarée avec la somme des modules mémoire
TRG_SOFTDEL_COMPUTER_CERGY	Propager la suppression logique aux composants liés
PRC_TRANSFER_COMPUTER	Transférer une machine entre sites avec COMMIT/ROLLBACK
FN_IP_IN_NETWORK	Tester l'appartenance d'une adresse IP à un sous-réseau CIDR
PKG_GLPI_UTILS	Regrouper les fonctions et procédures utiles dans un package maintenable

L'ensemble est regroupé dans un package PL/SQL. Cette organisation est plus propre qu'une collection de fonctions isolées : elle sépare la spécification publique du corps d'implémentation et facilite la maintenance dans un projet d'équipe.

7. Vues, vues matérialisées et base de données répartie

Les vues jouent un rôle central dans la lisibilité du modèle. La séparation physique des tables par site est utile pour l'architecture, mais elle ne doit pas compliquer les requêtes des utilisateurs. La vue V_ALL_COMPUTERS permet donc de reconstruire une vision globale du parc en agrégeant les machines de Cergy et de Pau avec UNION ALL.

Les vues d'inventaire enrichies, comme V_INVENTORY_CERGY et V_INVENTORY_PAU, simplifient l'accès aux informations métier. Elles regroupent les données machines avec des informations de contexte : utilisateur, système d'exploitation, localisation, fabricant, modèle ou entité. Elles évitent de réécrire des jointures complexes à chaque consultation.

Les vues matérialisées répondent à un autre besoin : optimiser les consultations plus coûteuses, notamment dans un contexte multi-sites. Une vue matérialisée stocke physiquement le résultat d'une requête. Cela peut réduire les recalculs et limiter les accès réseau. En contrepartie, les données peuvent être légèrement décalées selon le mode de rafraîchissement choisi.

```
CREATE OR REPLACE VIEW V_ALL_COMPUTERS AS
SELECT ... FROM COMPUTERS_CERGY
UNION ALL
SELECT ... FROM COMPUTERS_PAU;

CREATE MATERIALIZED VIEW MV_ALL_INVENTORY AS
SELECT ... FROM MV_INVENTORY_CERGY
UNION ALL
SELECT ... FROM MV_INVENTORY_PAU;
```

Pour la BDDR, l'architecture prévoit deux nœuds Oracle : un nœud Cergy et un nœud Pau. Les deux sites peuvent communiquer à l'aide de DATABASE LINK. Même si le déploiement physique dépend de l'environnement disponible, les scripts modélisent clairement cette séparation. Les vues matérialisées constituent alors une solution de réplication et de reporting entre sites.

8. Optimisation : clusters, index et plans d'exécution

L'optimisation ne repose pas sur un seul mécanisme. Le projet combine des clusters Oracle, des index adaptés et l'analyse des plans d'exécution. Les clusters rapprochent physiquement des lignes de tables différentes souvent jointes sur la même clé. Dans le projet, ils ciblent les associations ordinateur-port réseau et équipement réseau-port réseau.

Les index sont définis en fonction des requêtes attendues. Les index composites, comme IDX_CC_ENT_DEL sur entities_id et is_deleted, accélèrent les recherches de machines actives par site. Les index fonctionnels sur UPPER(name) facilitent les recherches insensibles à la casse. Les index bitmap sont proposés pour des colonnes à faible cardinalité comme is_deleted, ce qui est cohérent pour des usages principalement orientés lecture ou reporting.

Le projet évite l'indexation gratuite. Chaque index répond à une situation concrète : filtrer les machines d'une entité, exclure les éléments supprimés logiquement, rechercher un ordinateur par nom ou par numéro de série, ou accélérer les jointures fréquentes avec les composants.

```
CREATE INDEX IDX_CC_ENT_DEL ON COMPUTERS_CERGY(entities_id, is_deleted);
CREATE INDEX IDX_CP_ENT_DEL ON COMPUTERS_PAU(entities_id, is_deleted);
CREATE INDEX IDX_CC_NAME ON COMPUTERS_CERGY(UPPER(name));
CREATE BITMAP INDEX BIDX_CC_DEL ON COMPUTERS_CERGY(is_deleted);
```

Les tests de performance utilisent EXPLAIN PLAN et DBMS_XPLAN.DISPLAY. L'intérêt est de vérifier la stratégie réellement retenue par Oracle : full table scan, index range scan, coût estimé, accès par index ou accès via vue matérialisée. Cette approche permet de justifier les choix techniques au lieu de simplement affirmer qu'ils améliorent les performances.

9. Jeu de test et validation des performances

Pour valider la structure, le projet génère un jeu de test conséquent en PL/SQL. L'objectif est d'éviter une validation artificielle sur quelques lignes seulement. Le jeu de test simule un parc avec des utilisateurs, ordinateurs, équipements réseau, ports, adresses IP, VLANs et modules mémoire. Le volume annoncé est d'environ dix à quinze mille enregistrements selon les objets comptabilisés.

Objet généré	Volume indicatif
--------------	------------------

Utilisateurs	1 000
Ordinateurs Cergy	5 000
Ordinateurs Pau	3 000
Équipements réseau	500
Ports réseau	2 000
Adresses IP	environ 500
VLANs	50
Modules mémoire Cergy	2 000

Les tests de performance portent sur plusieurs cas représentatifs. Le filtrage des machines actives par entité permet de tester l'index composite sur `entities_id` et `is_deleted`. La recherche par numéro de série permet de vérifier l'intérêt d'un accès direct via contrainte d'unicité ou index. L'accès à `MV_ALL_INVENTORY` permet de démontrer l'apport des vues matérialisées pour les consultations globales.

Les résultats attendus sont une amélioration des plans d'exécution : certaines requêtes peuvent passer d'un parcours complet de table à un accès par index, et certaines requêtes globales peuvent s'appuyer sur des vues matérialisées au lieu de recalculer systématiquement les jointures. Le projet reste néanmoins prudent : pour une comparaison industrielle complète, il faudrait benchmarker l'ancien modèle et le nouveau modèle avec les mêmes données, le même serveur et les mêmes requêtes.

```
EXPLAIN PLAN SET STATEMENT_ID = 'TEST_INDEX' FOR
SELECT * FROM COMPUTERS_CERGY
WHERE entities_id = 1 AND is_deleted = 0;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

10. Bilan, limites et améliorations possibles

Le projet répond aux principaux attendus du sujet. Il présente une démarche de reverse engineering, une nouvelle structure Oracle, une utilisation concrète des concepts avancés, un jeu de test et des tests de performance. Il montre aussi que les choix techniques sont liés à des problèmes concrets : cohérence des données, gestion multi-sites, sécurité, optimisation et maintenabilité.

Cependant, certaines limites doivent être assumées. La BDDR est principalement modélisée dans les scripts : selon l'environnement disponible, elle peut ne pas être déployée sur deux serveurs Oracle physiques. La comparaison avec l'ancienne base GLPI est surtout structurelle et basée sur les plans d'exécution du nouveau modèle. Enfin, certains choix, comme les index bitmap ou les vues matérialisées en refresh périodique, devraient être validés plus finement dans un contexte de production réel.

Ces limites ne remettent pas en cause le projet, mais elles montrent les pistes d'amélioration possibles. Une version plus avancée pourrait déployer réellement deux instances Oracle, ajouter des materialized view logs pour tester le FAST REFRESH, comparer les performances avec une base GLPI/MySQL sur les mêmes données, et automatiser davantage les tests d'intégrité.

Point fort	Impact
Intégrité référentielle renforcée	Moins de risques de données orphelines
Séparation multi-sites	Architecture plus adaptée à Cergy et Pau
PL/SQL métier	Automatisation, audit et transactions
Vues et vues matérialisées	Accès simplifié et reporting plus efficace
Index et clusters	Optimisation ciblée des requêtes fréquentes

11. Conclusion

Ce mini-projet montre comment une base issue d'un modèle applicatif comme GLPI peut être repensée avec une logique Oracle plus structurée. Le travail réalisé ne vise pas à remplacer GLPI en production, mais à démontrer comment les concepts de bases de données avancées peuvent améliorer un périmètre concret : gestion du matériel, des utilisateurs, du réseau et des sites.

La nouvelle architecture apporte plusieurs améliorations : intégrité référentielle, transactions, tablespaces dédiés, rôles Oracle, profil de sécurité, index adaptés, clusters, vues, vues matérialisées, PL/SQL et logique de base répartie. Ces éléments renforcent la robustesse, la performance et la maintenabilité du modèle.

Le projet met également en évidence une logique professionnelle : partir d'un diagnostic, définir des choix techniques justifiés, tester la structure et expliquer les limites. C'est cette cohérence globale qui constitue la valeur principale du travail réalisé.

Sources utilisées

- Sujet du mini-projet GLPI — ING2 Bases de données avancées, CY Tech, année 2025-2026.
- Dépôt GitHub du projet : <https://github.com/Axelctrtr/bdd.git>
- Support oral fourni par l'équipe : support_oral_glpi_oracle.docx.
- Scripts SQL et PL/SQL du dépôt : 01_reverse_engineering.sql, 02_tablespaces_roles_users.sql, 03_ddl_tables_clusters_index.sql, 04_vues.sql, 05_plsql_triggers_procedures.sql, 06_jeu_de_test.sql, 07_tests_performance.sql.